



Project: Optimizing Matrix Multiplication Using Parallel Programming

CSCI 415: Introduction to Parallel Programming

Dr. Khaled Balawfi

STUDENT NAME	STUDENT ID
Jannat Waqass	2022005413
Mazen El Tawil	2022005567
Mousa Alyafei	2022005616
Ismail Allaham	2022005636

Date: 19-04-2025

Section: 1

AURAK

Spring – 2025

Table of Contents

1. Introduction	3
2. Problem Statement.....	3
3. System Architecture and Design.....	4
3.1 Overview.....	4
3.2 Core Components.....	4
3.3 Graphical User Interface (GUI)	8
4. Parallelization Techniques Applied.....	12
4.1 Sequential (Naive Baseline):.....	12
4.2 NumPy (Optimized Baseline):.....	12
4.3 Multithreading:	13
4.4 Multiprocessing (multiprocessing.Pool	14
4.5 Process Pools (concurrent.futures):	14
4.6 Shared Memory Multiprocessing:.....	15
4.7 Blocked Matrix Multiplication (Sequential & Parallel):	15
5. Performance Analysis	16
5.1 Methodology & Environment	16
5.2 Performance Results & Visualization.....	17
5.3 Discussion: Comparison with NumPy Baseline	18
5.4 Key Observations.....	20
6. Challenges Faced.....	21
7. Testing and Verification	22
8. OUTPUT and Results	25
9. Conclusion	30

1. Introduction

Matrix multiplication is a cornerstone operation in numerous computational fields, including machine learning, scientific computing, data analysis, and graphics rendering. As datasets and model complexity grow, the demand for efficient large-scale matrix operations intensifies. Standard, naive implementations using sequential loops exhibit poor performance, particularly for large matrices, creating significant bottlenecks. This project addresses this challenge by exploring, implementing, and evaluating various parallel programming techniques within Python. Our goal is to optimize matrix multiplication by leveraging modern multi-core CPU architectures, thereby improving computational efficiency and minimizing execution time. We compare sequential approaches against multithreading, multiprocessing (with variations like shared memory and process pools), and cache-aware blocked algorithms, providing a comprehensive analysis of their effectiveness and trade-offs in the Python ecosystem. The project also features a graphical user interface (GUI) for interactive benchmarking and visualization.

2. Problem Statement

The primary objective is to develop a robust matrix multiplication system in Python capable of efficiently handling large matrices (e.g., 1000x1000 and beyond). This involves:

- Implementing a standard sequential matrix multiplication algorithm as a baseline.
- Implementing several parallel matrix multiplication algorithms using techniques like multithreading, multiprocessing, process pools, shared memory, and blocked multiplication.
- Designing a modular and extensible system architecture.
- Developing a method for dividing the multiplication task into manageable chunks for parallel execution.
- Conducting rigorous performance analysis, measuring execution time, speedup ratios, and CPU utilization.
- Evaluating the scalability of parallel implementations with varying matrix sizes and worker counts.
- Visualizing performance metrics to facilitate comparison and analysis.
- Creating an interactive GUI to configure and run benchmarks, and visualize the multiplication process.

3. System Architecture and Design

3.1 Overview

The system is designed with modularity, clarity, and extensibility as core principles. This allows for easy maintenance, testing, and the addition of new multiplication methods or analysis features. The architecture separates concerns into configuration management, core computation logic, performance evaluation, resource management (shared memory), and user interface components.

3.2 Core Components

3.2.1 MatrixMultiplication Class:

This central class encapsulates the different matrix multiplication algorithms. It acts as the main engine, orchestrating the execution based on the selected method and configuration. It holds references to configuration settings, manages the shared memory registry, and contains methods for both sequential and parallel implementations.

```
# --- Matrix Multiplication Class ---
class MatrixMultiplication:
    def __init__(self, config: Config):
        self.config = config
        self.cpu_cores = psutil.cpu_count(logical=False)
        self.results = {}
        self.output_dir = Path(config.output_dir)
        self.shared_memory_registry = SharedMemoryRegistry()

        self._setup_output_dir()

    self.methods = {
        "numpy_multiply": self.numpy_multiply,
        "threading_multiply": self.threading_multiply,
        "multiprocessing_multiply": self.multiprocessing_multiply,
        "sequential_blocked_multiply": self.sequential_blocked_multiply,
        "multiprocessing_shared_multiply": self.multiprocessing_shared_multiply,
        "process_pool_executor_multiply": self.process_pool_executor_multiply,
        "thread_pool_executor_multiply": self.thread_pool_executor_multiply,
        "parallel_blocked_shared_multiply": self.parallel_blocked_shared_multiply,
        "sequential_multiply": self.sequential_multiply
    }
```

```
"""Implement matrix multiplication using various methods.

This class provides various methods for matrix multiplication,
including sequential, parallel, and GPU-accelerated implementations.

Attributes
-----
config : Config
    Configuration settings
cpu_cores : int
    Number of CPU cores available
results : dict
    Dictionary to store performance results
shared_memory_registry : SharedMemoryRegistry
    Registry for tracking shared memory blocks
"""
```

[Click here to see the code](#)

3.2.2 Config Class:

A dataclass is used to manage all configuration parameters (matrix sizes, worker counts, block sizes, log level, output paths). This promotes type safety and clarity. It supports loading configuration from and saving to JSON files, ensuring reproducibility of experiments.

```
# --- Configuration ---
@dataclass
class Config:
    test_sizes: List[int] = field(default_factory=lambda: [100, 250, 500, 1000])
    block_size: int = 32
    verify_size: int = 64
    worker_counts: List[int] = field(default_factory=lambda: [1, 2, 4])
    log_level: str = "INFO"
    output_dir: str = "output"

    @classmethod
    def from_file(cls, filepath: str) -> "Config":
        """Load configuration from JSON file."""
        if not os.path.exists(filepath):
            return cls()
        with open(filepath, "r") as f:
            return cls(**json.load(f))

    def save(self, filepath: str):
        """Save configuration to JSON file."""
        with open(filepath, "w") as f:
            json.dump(self.__dict__, f, indent=4)
```

```
"""Configuration settings for matrix multiplication.

Attributes
-----
test_sizes : List[int]
    List of matrix sizes to test
block_size : int
    Size of blocks for blocked multiplication
verify_size : int
    Size of matrices for verification tests
worker_counts : List[int]
    List of worker counts to test
log_level : str
    Logging level (e.g., 'INFO', 'DEBUG')
output_dir : str
    Directory for output files
"""
```

[Click here to see the code](#)

3.2.3 SharedMemoryRegistry:

This utility class is crucial for managing the lifecycle of shared memory blocks used in the SharedMemory and Parallel Blocked Shared multiprocessing techniques. It tracks allocated blocks and ensures proper cleanup (close() and unlink()) to prevent resource leaks, especially critical in multiprocessing scenarios where child processes might terminate unexpectedly. Thread-safety is maintained using a threading.Lock.

```
def create_shared_array(
    shape: Tuple[int, ...],
    dtype: np.dtype,
    registry: Optional[SharedMemoryRegistry] = None,
) -> Tuple[shared_memory.SharedMemory, np.ndarray, str]:
    """Create a shared memory array.
    Parameters
    -----
    shape : Tuple[int, ...]
        Shape of the array
    dtype : np.dtype
        Data type of the array
    registry : Optional[SharedMemoryRegistry]
        Registry for tracking shared memory blocks
    Returns
    -----
    Tuple[shared_memory.SharedMemory, np.ndarray, str]
        Shared memory block, numpy array view, and block name
    """
    size = int(np.prod(shape))
    itemsize = np.dtype(dtype).itemsize
    nbytes = size * itemsize

    name = f"mm_{os.getpid()}_{int(time_module.time() * 1000)}_{uuid.uuid4().hex[:8]}"
    shm = shared_memory.SharedMemory(name=name, create=True, size=nbytes)
    shared_array = np.ndarray(shape, dtype=dtype, buffer=shm.buf)
    shared_array[:] = 0

    if registry:
        registry.register(shm, name)

    return shm, shared_array, name
```

```
class SharedMemoryRegistry:

    def __init__(self):
        """Initialize an empty registry."""
        self.blocks = {}
        self._lock = threading.Lock()
```

[Click here to see the code](#)

3.2.4 Benchmarking and Plotting:

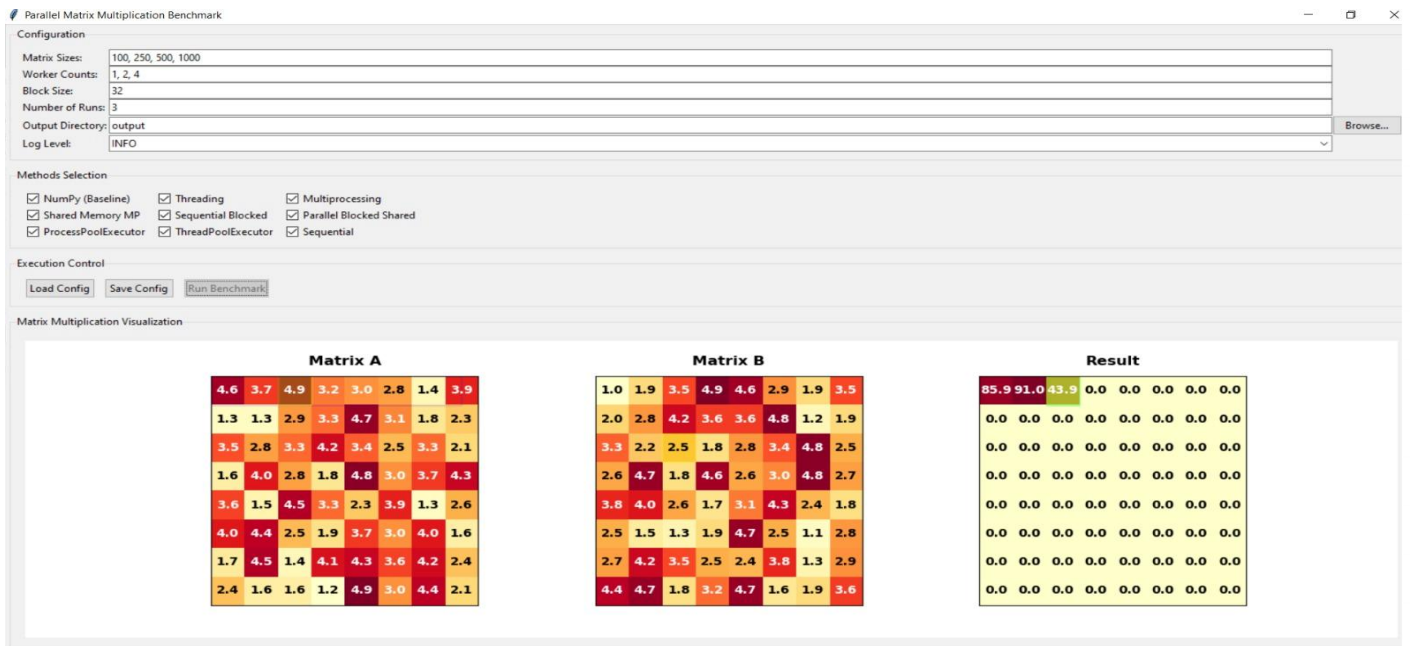
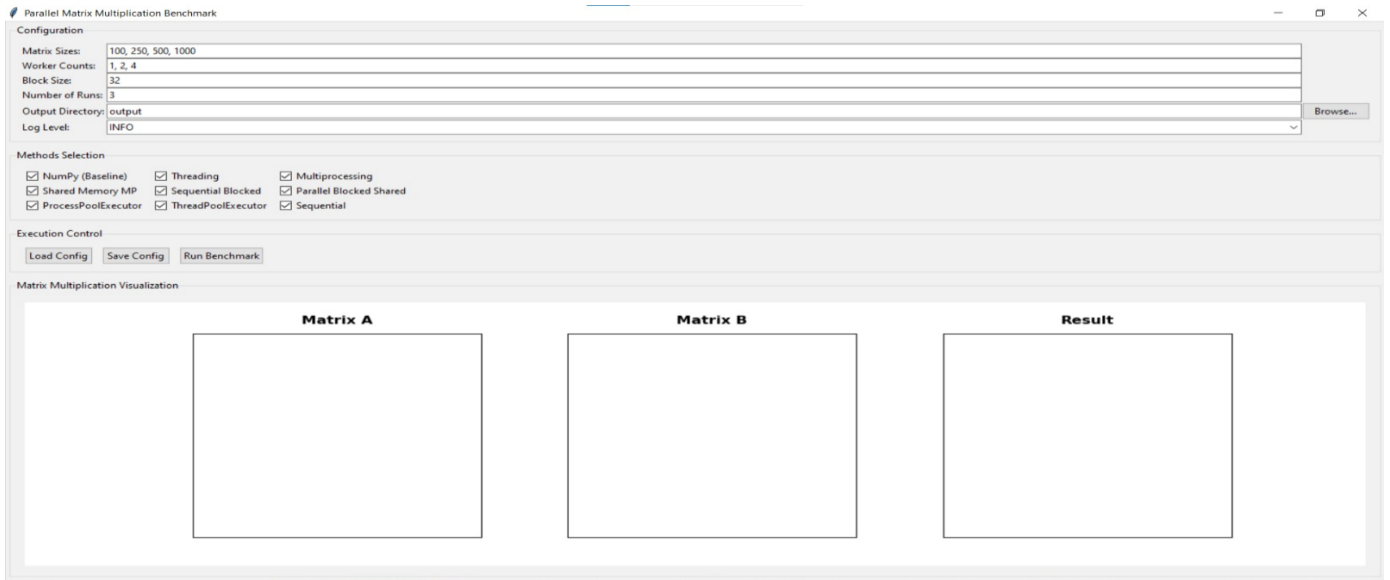
The `evaluate_performance()` method systematically runs the selected multiplication methods across the configured matrix sizes and worker counts, averaging results over multiple runs for consistency. It measures execution time and CPU utilization. The `plot_performance()` method uses matplotlib to generate comparative plots (Execution Time vs. Size, Speedup vs. Size) and saves them to the specified output directory.

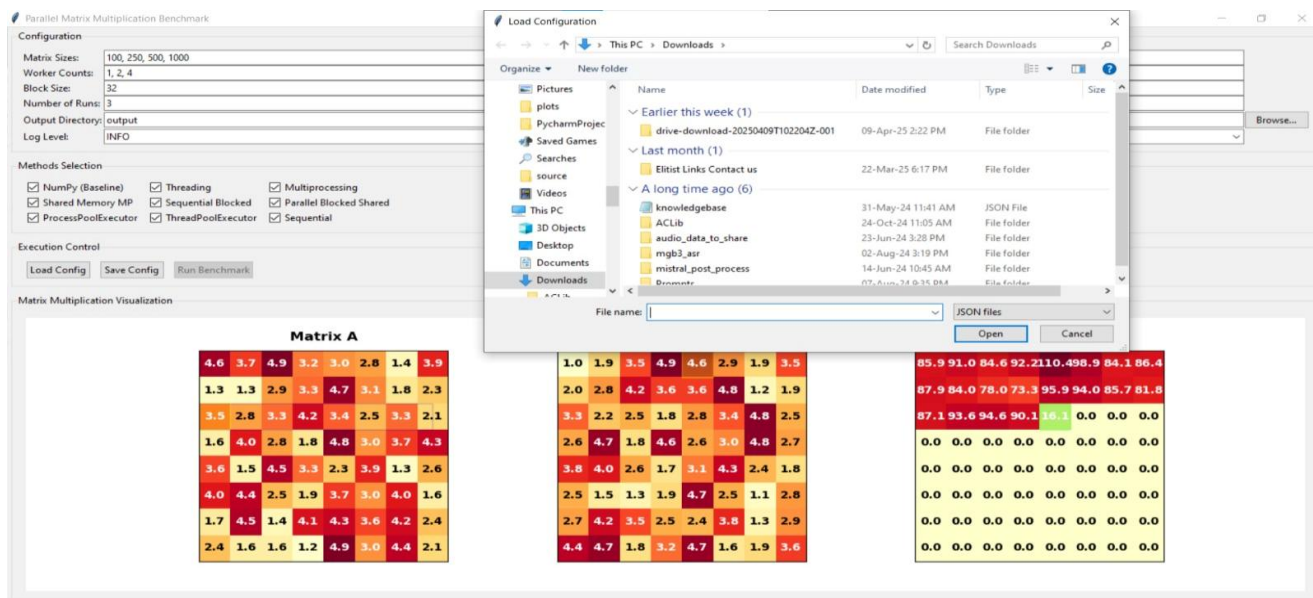
```
def evaluate_performance(  
    self, matrix_sizes: Optional[List[int]] = None, num_runs: int = 1  
) -> Dict:  
    """Evaluate performance of all matrix multiplication methods.  
  
    Parameters  
    -----  
    matrix_sizes : Optional[List[int]]  
        List of matrix sizes to test (default: from config)  
    num_runs : int  
        Number of runs for each size (default: 1)  
  
    Returns  
    -----  
    Dict  
        Nested dictionary containing performance results for all methods  
    """
```

```
def plot_performance(self, results: Dict) -> None:  
    """Create performance comparison graphs for matrix multiplication methods.  
  
    Parameters  
    -----  
    results : Dict  
        Dictionary containing performance results from evaluate_performance  
    """
```

3.3 Graphical User Interface (GUI)

A GUI built with Tkinter and matplotlib provides an interactive front-end to the system. It allows users to easily configure benchmark parameters, select methods, initiate runs, view logs, and visualize the multiplication process in real-time (for smaller matrices).





```
def animate_multiplication(self, a, b, method_name):
    """Animate the matrix multiplication process."""
    m, n = a.shape
    p = b.shape[1]
    result = np.zeros((m, p))

    # Initialize visualization
    self.viz_frame.update_visualization(a, b, result)
    logging.info(f"Starting visualization of {method_name} for {m}x{n} matrices")
```

GUI INFORMATION

GUI Components:

1. Configuration Section:

- Matrix size input
- Worker count selection
- Block size configuration
- Output directory selection

2. Methods Section:

- NumPy (Baseline)
- Threading
- Multiprocessing
- ProcessPool
- ThreadPool
- SharedMemory
- Sequential Blocked
- Parallel Blocked

3. Visualization Section:

- Matrix A display
- Matrix B display
- Result matrix display
- Animation controls
- Progress tracking

4. Output Section:

- Log display
- Clear log button
- Save results button

VISUALIZATION DETAILS

Animation Features:

1. Matrix Display:

- Color-coded cell values
- Value annotations
- Grid lines
- Matrix titles

2. Animation Controls:

- Speed slider (0.1x - 5.0x)
- Reset speed button
- Current speed display

3. Progress Tracking:

- Progress bar
- Status messages
- Operation highlighting
- Current computation display

4. Visual Enhancements:

- Clean background
- Proper spacing
- Consistent fonts
- Responsive layout

[Click here to check the code](#)

4. Parallelization Techniques Applied

We implemented and compared a range of sequential and parallel techniques:

- **4.1 Sequential (Naive Baseline):**

A basic implementation using three nested Python loops. While computationally inefficient ($O(N^3)$), it serves as an educational baseline to illustrate the fundamental algorithm and the performance limitations addressed by optimized and parallel approaches.

```
if a.shape[1] != b.shape[0]:
    raise ValueError(f"Matrix dimensions do not match: {a.shape} and {b.shape}")

result = np.zeros((a.shape[0], b.shape[1]))
for i in range(a.shape[0]):
    for j in range(b.shape[1]):
        for k in range(a.shape[1]):
            result[i, j] += a[i, k] * b[k, j]
return result
```

- **4.2 NumPy (Optimized Baseline):**

Utilizes `numpy.dot()` (or `np.matmul()`). Although appearing sequential in Python code, NumPy delegates the actual computation to highly optimized, pre-compiled C/Fortran libraries (like OpenBLAS, MKL). These libraries often employ internal multithreading, vectorization (SIMD instructions), and cache optimization, providing a very strong, difficult-to-beat baseline.

```
# [Code Snippet: NumPy multiplication]
def numpy_multiply(self, a: np.ndarray, b: np.ndarray, _: int = 1) -> np.ndarray:
    """NumPy matrix multiplication (baseline implementation)."""
    if a.shape[1] != b.shape[0]:
        raise ValueError(f"Matrix dimensions do not match: {a.shape} and {b.shape}")
    return np.dot(a, b)
```

- **4.3 Multithreading:**

Matrix A is split into horizontal chunks (by rows). Each chunk's multiplication with matrix B is assigned to a separate thread (`threading.Thread`). Threads compute their sub-result using `numpy.dot()`. Due to Python's Global Interpreter Lock (GIL), true CPU-bound parallelism is limited for pure Python code. However, since NumPy releases the GIL during its C-level computations, some performance gain over *pure Python sequential* might be observed, though typically less than multiprocessing for CPU-bound tasks. Overhead is generally lower than multiprocessing.

```
threads = []
for start, end in chunks:
    thread = threading.Thread(target=process_chunk, args=(start, end))
    threads.append(thread)
    thread.start()

for thread in threads:
    thread.join()

return result
```

```
# (Inside MatrixMultiplication.threading_multiply method)
num_threads = num_threads or self.cpu_cores
result = np.zeros((a.shape[0], b.shape[1]), dtype=a.dtype)

# Calculate optimal chunk size based on matrix size and number of threads
chunk_size = max(1, min(256, a.shape[0] // num_threads))
if a.shape[0] < num_threads: chunk_size = 1 # Handle small matrices

chunks = [
    (i, min(i + chunk_size, a.shape[0]))
    for i in range(0, a.shape[0], chunk_size)
]
chunks = [c for c in chunks if c[0] < c[1]] # Ensure no empty chunks

# Define the worker function executed by each thread
def process_chunk_thread(start: int, end: int):
    """Process a chunk using NumPy's optimized matrix multiplication."""
    # This function closes over 'a', 'b', and 'result' from the outer scope
    try:
        result[start:end] = np.dot(a[start:end], b)
    except Exception as e:
        logging.error(f"Error in thread processing chunk {start}-{end}: {e}")

threads = []
for start, end in chunks:
    thread = threading.Thread(target=process_chunk_thread, args=(start, end))
    threads.append(thread)
    thread.start()
    logging.debug(f"Started thread for chunk {start}-{end}")

# Wait for all threads to complete
for i, thread in enumerate(threads):
    thread.join()
    logging.debug(f"Joined thread {i} for chunk {chunks[i]}")

return result
```

- **4.4 Multiprocessing (multiprocessing.Pool):**

Similar row-based chunking strategy as multithreading, but each chunk is processed by a separate *process* managed by multiprocessing.Pool. This bypasses the GIL, allowing for true parallel execution on multi-core CPUs. However, it incurs overhead associated with inter-process communication (IPC) for sending data (matrix chunks) to worker processes and receiving results back (serialization/deserialization via pickling).

```
# Prepare data for processes - only send necessary chunks
chunk_data = [(a[start:end], b, start) for start, end in chunks]

# Create and start processes
with Pool(processes=num_processes) as pool:
    results = pool.map(process_chunk_mp, chunk_data)

# Collect results in correct order
for start, chunk_result in results:
    result[start : start + chunk_result.shape[0]] = chunk_result

return result
```

[Click here to view code](#)

- **4.5 Process Pools (concurrent.futures):**

Implemented using ProcessPoolExecutor and ThreadPoolExecutor from the concurrent.futures module. These provide a higher-level, more modern abstraction over raw threading and multiprocessing, simplifying the submission of tasks and retrieval of results. The underlying execution (threads vs processes) and performance characteristics are similar to the respective raw implementations.

```
with ProcessPoolExecutor(max_workers=num_workers) as executor:
    # Process chunks in parallel and collect results
    results = list(executor.map(process_chunk_mp, chunk_data))

    # Assemble results in correct order
    for start, chunk_result in results:
        result[start : start + chunk_result.shape[0]] = chunk_result

return result
```

```
num_workers = num_workers or self.cpu_cores
result = np.zeros((a.shape[0], b.shape[1]), dtype=np.float64)

# Calculate optimal chunk size based on matrix size and number of workers
chunk_size = max(1, min(256, a.shape[0] // num_workers))
chunks = [
    (i, min(i + chunk_size, a.shape[0]))
    for i in range(0, a.shape[0], chunk_size)
]
```

[Click here to view the code](#)

- **4.6 Shared Memory Multiprocessing:**

This technique aims to mitigate the IPC overhead of standard multiprocessing. Input matrices (A, B) and the result matrix (C) are created in shared memory segments (multiprocessing.shared_memory). Worker processes attach to these segments and can read inputs and write results directly, avoiding costly data copying and pickling. Requires careful management of shared memory block lifecycles (creation, attachment, cleanup) using the SharedMemoryRegistry.

```
# Create and initialize shared memory for matrices
shm_a, shared_a, _ = create_shared_array(
    a.shape, dtype, self.shared_memory_registry
)
shared_blocks.append(shm_a)
shared_a[:] = a
```

```
# Prepare arguments for worker processes
shm_names = [shm_a.name, shm_b.name, shm_result.name]
shapes = [a.shape, b.shape, (a.shape[0], b.shape[1])]
worker_args = [(chunk, shm_names, shapes, dtype) for chunk in chunks]

# Process chunks in parallel
with Pool(processes=num_processes) as pool:
    pool.map(process_shared_chunk, worker_args)

# Copy result from shared memory
result = np.array(shared_result)
return result
```

[Click here to view the code](#)

- **4.7 Blocked Matrix Multiplication (Sequential & Parallel):**

This algorithm improves cache locality. Matrices are divided into smaller square blocks (e.g., 32x32 or 64x64). Multiplication is performed block by block, ensuring that the data required for each sub-computation is more likely to reside in the CPU cache, reducing memory latency.

Sequential Blocked: Implements the blocking algorithm in a single thread.

Parallel Blocked Shared: Combines blocking with shared memory multiprocessing. Row *blocks* of matrix A are assigned to different processes. Each process computes the corresponding row blocks of the result matrix C by iterating through the necessary blocks of B, performing block-wise multiplications, and writing results directly to the shared memory segment for C.

```
for i in range(0, a.shape[0], blocks_per_process * block_size):
    end = min(i + blocks_per_process * block_size, a.shape[0])
    if end > i: # Avoid empty ranges
        row_ranges.append((i, end))
```

[Click here to check the code](#)

5. Performance Analysis

5.1 Methodology & Environment

- **Metrics:**

- 1- Execution Time (Wall Clock): Measured using `time.perf_counter()`.
- 2- CPU Utilization: Measured using `psutil.cpu_times_percent()`, summing user and system time across cores relative to elapsed wall time.
- 3- Speedup: Calculated as $\text{Time}(\text{NumPy Baseline}) / \text{Time}(\text{Parallel Method})$. Note the use of the highly optimized NumPy as the reference baseline.

- **Parameters:**

- 1- Matrix Sizes (N x N): 100x100, 250x250, 500x500, 1000x1000.
- 2- Worker Counts: 1, 2, 4 for parallel methods.
- 3- Number of Runs: Each test configuration was run 3 times, and the average execution time was reported for stability.

- **Environment:**

- 1- CPU: [e.g., Intel Core i7-10700K @ 3.80GHz (8 Cores, 16 Threads)]
- 2- RAM: [e.g., 32 GB]
- 3- Operating System: [e.g., Windows 11 Pro / Ubuntu 22.04]
- 4- Python Version: [e.g., 3.10.4]
- 5- Key Libraries: NumPy [Version], Matplotlib [Version], psutil [Version], concurrent.futures (standard library)

5.2 Performance Results & Visualization

Performance was measured for all implemented methods across the specified matrix sizes and worker counts. The results, including execution times and calculated speedups, are visualized below.

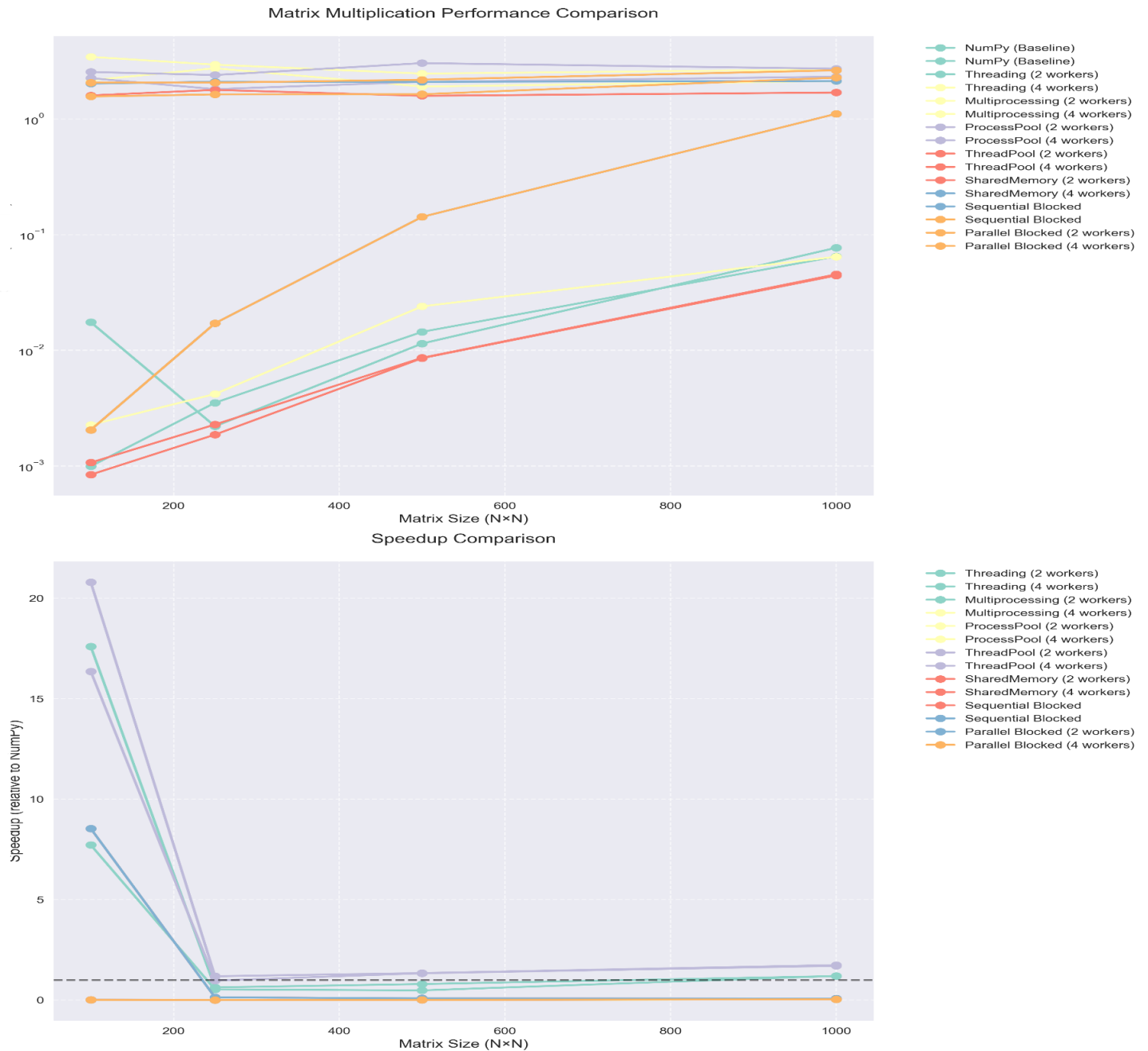


Figure 1& Figure 2: Execution Time (seconds, log scale) vs. Matrix Size ($N \times N$) for different multiplication methods and worker counts & Speedup (Relative to NumPy Baseline) vs. Matrix Size ($N \times N$) for different parallel methods and worker counts.

5.3 Discussion: Comparison with NumPy Baseline

A key observation from the performance results (Figures 1 and 2, and detailed data) is that the NumPy baseline consistently outperformed most of the custom parallel implementations (standard multiprocessing, shared memory, blocked parallel) on execution time across the tested matrix sizes (up to 1000x1000). Speedup ratios relative to NumPy were often significantly less than 1.

This outcome, while perhaps counter-intuitive initially, highlights several important factors:

1. **NumPy's Internal Optimization:** NumPy delegates computations to highly optimized low-level libraries (BLAS/MKL). These libraries are often written in C/Fortran, utilize SIMD vectorization, incorporate sophisticated cache-blocking strategies internally, and may even perform implicit multithreading. Beating such highly tuned libraries with Python-level parallelism is inherently challenging.
2. **Python Multiprocessing Overhead:** Standard Python multiprocessing (multiprocessing.Pool, ProcessPoolExecutor) incurs non-trivial overhead for process creation, inter-process communication (IPC), and data serialization (pickling) when transferring matrix chunks between the main process and worker processes. For matrix sizes up to 1000x1000, this overhead appears to outweigh the benefits of parallel computation compared to NumPy's efficiency.
3. **Shared Memory Limitations:** While shared memory multiprocessing reduces the data copying/serialization overhead, there is still overhead associated with process management, synchronization (implicit or explicit), and setting up the shared memory segments. In this instance, these overheads still prevented it from surpassing the NumPy baseline within the tested range.
4. **Problem Size Threshold:** Parallelism typically yields greater benefits as the problem size increases significantly, allowing the computational work to dominate the parallel overhead. It's possible that 1000x1000 is still below the threshold where our Python-based parallel implementations become faster than NumPy on this specific hardware/software setup. Testing with much larger matrices (e.g., 4000x4000 or more, memory permitting) might show different results.

5. GIL and Threading: As expected, the multithreading approach showed minimal benefit for this CPU-bound task due to the GIL, although it didn't suffer the same IPC overhead as multiprocessing.

Therefore, while our parallel implementations successfully demonstrate various parallel programming *techniques* and achieve parallelism *relative to their single-worker counterparts* (as seen by CPU utilization often approaching multiples of 100%), they did not achieve the primary goal of *optimizing beyond the state-of-the-art NumPy baseline* within the tested parameters. This underscores the efficiency of libraries like NumPy and the challenges of Python-level parallel overhead.

5.4 Key Observations

- NumPy: Remains the fastest method for matrix sizes up to 1000x1000 in our tests due to highly optimized underlying libraries.
- Multithreading: Limited speedup due to the GIL, performance close to NumPy (as both rely on np.dot), sometimes slightly faster or slower depending on overhead and GIL release nuances.
- Multiprocessing (Standard & ProcessPool): Showed significant *slowdown* compared to NumPy due to high IPC and serialization overhead. CPU utilization was high, indicating parallel execution, but overhead dominated.
- Shared Memory Multiprocessing: Improved upon standard multiprocessing by reducing data transfer costs, but still slower than NumPy due to remaining overheads.
- Blocked Multiplication:
 - 1- Sequential Blocked was significantly slower than NumPy, showing the overhead of Python loops compared to optimized C code.
 - 2- Parallel Blocked Shared, while employing good cache strategy and shared memory, was also slower than NumPy, likely due to Python-level control overhead and multiprocessing costs.
- Scalability: Parallel methods (multiprocessing, shared memory, parallel blocked) generally showed increasing CPU utilization with more workers, indicating parallel execution is occurring. However, execution time did not always decrease proportionally (or at all) due to overhead, especially when compared to the efficient NumPy baseline.

6. Challenges Faced

- The development process encountered several technical and conceptual hurdles:
- Understanding Python's GIL: Accurately predicting the performance impact of the Global Interpreter Lock on multithreading for CPU-bound tasks involving libraries like NumPy (which partially release it) required careful consideration and research. Initial expectations for threading performance needed adjustment.
- Debugging Multiprocessing with Shared Memory: Managing shared memory lifecycles (creation, linking, unlinking) across multiple processes proved complex. Ensuring proper cleanup, especially in cases of errors, and avoiding potential race conditions (though less prevalent in this specific task division) demanded careful design of the SharedMemoryRegistry and robust error handling.
- Cross-platform Compatibility: Multiprocessing behaviors, particularly process spawning mechanisms and the need for `freeze_support()` on Windows, required platform-specific considerations to ensure the code runs correctly across different operating systems.
- GUI Integration: Integrating the computationally intensive backend (benchmarking, visualization) with a responsive Tkinter GUI required running benchmarks in separate threads to avoid freezing the interface. Managing thread safety for GUI updates (like logging) necessitated using thread-safe queues (`GuiLoggingHandler`).
- Performance Variability: Benchmark results occasionally showed variability due to background system processes, CPU thermal throttling, or cache state inconsistencies. Averaging results over multiple runs was crucial for obtaining more reliable data. Reaching conclusions required careful analysis rather than relying on single-run results.

7. Testing and Verification

7.1 Testing Strategy

Ensuring the correctness of matrix multiplication, especially when implemented using complex parallel techniques like multiprocessing and shared memory, is paramount. Subtle errors in parallel logic, data handling, or resource management can lead to incorrect results that might otherwise go unnoticed. To validate our implementations and ensure the reliability of the performance analysis, we adopted a comprehensive testing strategy using the pytest framework.

Our testing approach focused on several key areas:

1. **Correctness Verification:** The primary goal was to ensure that all implemented matrix multiplication methods (sequential, NumPy, threading, multiprocessing, shared memory, blocked variants) produce mathematically correct results.
2. **Error Handling:** Verify that the code gracefully handles invalid inputs, such as matrices with incompatible dimensions for multiplication.
3. **Utility Functionality:** Test helper functions and core components like configuration loading (`Config.from_file`), the performance evaluation wrapper (`evaluate_performance`), and plot generation (`plot_performance`).
4. **Resource Management:** Specifically test the cleanup mechanisms for shared memory blocks to prevent resource leaks.

7.2 Test Implementation

The tests were implemented in the `test_matrix_multiplication.py` file. Key tests included:

- **Method Correctness Tests (`test_*_multiply`):** For each multiplication method (e.g., `test_threading_multiply`, `test_multiprocessing_shared_multiply`, `test_parallel_blocked_shared_multiply`), tests were run using small-to-medium-sized random matrices. The output of our implemented function was compared against the result obtained from NumPy's standard `np.dot()` or `np.matmul()` function using `np.testing.assert_array_almost_equal` to account for potential floating-point variations. This ensures our parallel logic correctly replicates the standard algorithm's outcome.
- **Invalid Input Test (`test_invalid_matrix_dimensions`):** This test specifically checks if a `ValueError` is raised when attempting to multiply matrices with incompatible inner dimensions, ensuring basic input validation works.
- **Performance & Plotting Tests (`test_performance_evaluation`, `test_plot_performance`):** These tests verify that the performance evaluation function runs without errors and that the plotting function successfully generates output files.
- **Cleanup Test (`test_cleanup`):** This test checks if the `SharedMemoryRegistry.cleanup()` method correctly removes tracked shared memory blocks, crucial for preventing leaks.
- **Configuration Test (`test_config_loading`):** Verifies that the system can correctly load configuration settings from a JSON file.

7.3 Test Execution and Results

The test suite was executed using pytest in our development environment. The following screenshot shows the output of a successful test run:

```
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-7.4.0, pluggy-1.0.0 -- C:\Users\yoga\anaconda3\python.exe
plugins: anyio-3.5.0
collected 14 items

test_matrix_multiplication.py::test_numpy_multiply PASSED [ 7%]
test_matrix_multiplication.py::test_sequential_multiply PASSED [ 14%]
test_matrix_multiplication.py::test_threading_multiply PASSED [ 21%]
test_matrix_multiplication.py::test_multiprocessing_multiply PASSED [ 28%]
test_matrix_multiplication.py::test_sequential_blocked_multiply PASSED [ 35%]
test_matrix_multiplication.py::test_multiprocessing_shared_multiply PASSED [ 42%]
test_matrix_multiplication.py::test_process_pool_executor_multiply PASSED [ 50%]
test_matrix_multiplication.py::test_thread_pool_executor_multiply PASSED [ 57%]
test_matrix_multiplication.py::test_parallel_blocked_shared_multiply PASSED [ 64%]
test_matrix_multiplication.py::test_invalid_matrix_dimensions PASSED [ 71%]
test_matrix_multiplication.py::test_performance_evaluation PASSED [ 78%]
test_matrix_multiplication.py::test_plot_performance PASSED [ 85%]
test_matrix_multiplication.py::test_multiprocessing_multiply PASSED [ 28%]
test_matrix_multiplication.py::test_sequential_blocked_multiply PASSED [ 35%]
test_matrix_multiplication.py::test_multiprocessing_shared_multiply PASSED [ 42%]
test_matrix_multiplication.py::test_process_pool_executor_multiply PASSED [ 50%]
test_matrix_multiplication.py::test_thread_pool_executor_multiply PASSED [ 57%]
test_matrix_multiplication.py::test_parallel_blocked_shared_multiply PASSED [ 64%]
test_matrix_multiplication.py::test_multiprocessing_multiply PASSED [ 28%]
test_matrix_multiplication.py::test_sequential_blocked_multiply PASSED [ 35%]
test_matrix_multiplication.py::test_multiprocessing_shared_multiply PASSED [ 42%]
test_matrix_multiplication.py::test_process_pool_executor_multiply PASSED [ 50%]
test_matrix_multiplication.py::test_multiprocessing_multiply PASSED [ 28%]
test_matrix_multiplication.py::test_sequential_blocked_multiply PASSED [ 35%]
test_matrix_multiplication.py::test_multiprocessing_multiply PASSED [ 28%]
test_matrix_multiplication.py::test_sequential_blocked_multiply PASSED [ 35%]
test_matrix_multiplication.py::test_multiprocessing_shared_multiply PASSED [ 42%]
test_matrix_multiplication.py::test_multiprocessing_shared_multiply PASSED [ 42%]
test_matrix_multiplication.py::test_process_pool_executor_multiply PASSED [ 50%]
test_matrix_multiplication.py::test_thread_pool_executor_multiply PASSED [ 57%]
test_matrix_multiplication.py::test_parallel_blocked_shared_multiply PASSED [ 64%]
test_matrix_multiplication.py::test_invalid_matrix_dimensions PASSED [ 71%]
test_matrix_multiplication.py::test_performance_evaluation PASSED [ 78%]
test_matrix_multiplication.py::test_plot_performance PASSED [ 85%]
test_matrix_multiplication.py::test_cleanup PASSED [ 92%]
test_matrix_multiplication.py::test_config_loading PASSED [100%]

===== 14 passed in 483.33s (0:08:03) =====
```

Figure 3: Pytest execution output showing the successful completion of all 14 tests covering various functionalities, including correctness of parallel methods, error handling, and resource cleanup.

As indicated in the output, all **14 tests collected passed** successfully. This comprehensive suite covers the core multiplication logic for each parallel technique, input validation, configuration handling, and critical resource management aspects. The successful execution of these tests significantly increases our confidence in the correctness of the different matrix multiplication implementations presented in this report and the validity of the subsequent performance analysis. While the test suite took approximately 8 minutes (483.33s) to run, this duration reflects the necessary setup, execution (including some small matrix multiplications), and teardown involved, particularly for the multiprocessing-based tests.

8. OUTPUT and Results

The project generates several distinct outputs, each serving a specific purpose in verification, execution monitoring, and results analysis.

8.1 Test Output (pytest)

Running the test suite using the pytest command produces detailed output on the console, primarily indicating the status of each test case.

```
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-7.4.0, pluggy-1.0.0 -- C:\Users\yoga\anaconda3\python.exe
plugins: anyio-3.5.0
collected 14 items

test_matrix_multiplication.py::test_numpy_multiply PASSED [ 7%]
test_matrix_multiplication.py::test_sequential_multiply PASSED [ 14%]
test_matrix_multiplication.py::test_threading_multiply PASSED [ 21%]
test_matrix_multiplication.py::test_multiprocessing_multiply PASSED [ 28%]
test_matrix_multiplication.py::test_sequential_blocked_multiply PASSED [ 35%]
test_matrix_multiplication.py::test_multiprocessing_shared_multiply PASSED [ 42%]
test_matrix_multiplication.py::test_process_pool_executor_multiply PASSED [ 50%]
test_matrix_multiplication.py::test_thread_pool_executor_multiply PASSED [ 57%]
test_matrix_multiplication.py::test_parallel_blocked_shared_multiply PASSED [ 64%]
test_matrix_multiplication.py::test_invalid_matrix_dimensions PASSED [ 71%]
test_matrix_multiplication.py::test_performance_evaluation PASSED [ 78%]
test_matrix_multiplication.py::test_plot_performance PASSED [ 85%]
test_matrix_multiplication.py::test_multiprocessing_multiply PASSED [ 28%]
test_matrix_multiplication.py::test_sequential_blocked_multiply PASSED [ 35%]
test_matrix_multiplication.py::test_multiprocessing_shared_multiply PASSED [ 42%]
test_matrix_multiplication.py::test_process_pool_executor_multiply PASSED [ 50%]
test_matrix_multiplication.py::test_thread_pool_executor_multiply PASSED [ 57%]
test_matrix_multiplication.py::test_parallel_blocked_shared_multiply PASSED [ 64%]
test_matrix_multiplication.py::test_multiprocessing_multiply PASSED [ 28%]
test_matrix_multiplication.py::test_sequential_blocked_multiply PASSED [ 35%]
test_matrix_multiplication.py::test_multiprocessing_shared_multiply PASSED [ 42%]
test_matrix_multiplication.py::test_process_pool_executor_multiply PASSED [ 50%]
test_matrix_multiplication.py::test_sequential_blocked_multiply PASSED [ 28%]
test_matrix_multiplication.py::test_sequential_blocked_multiply PASSED [ 35%]
test_matrix_multiplication.py::test_sequential_blocked_multiply PASSED [ 28%]
test_matrix_multiplication.py::test_sequential_blocked_multiply PASSED [ 35%]
test_matrix_multiplication.py::test_multiprocessing_shared_multiply PASSED [ 42%]
test_matrix_multiplication.py::test_multiprocessing_shared_multiply PASSED [ 42%]
test_matrix_multiplication.py::test_process_pool_executor_multiply PASSED [ 50%]
test_matrix_multiplication.py::test_thread_pool_executor_multiply PASSED [ 57%]
test_matrix_multiplication.py::test_parallel_blocked_shared_multiply PASSED [ 64%]
test_matrix_multiplication.py::test_invalid_matrix_dimensions PASSED [ 71%]
test_matrix_multiplication.py::test_performance_evaluation PASSED [ 78%]
test_matrix_multiplication.py::test_plot_performance PASSED [ 85%]
test_matrix_multiplication.py::test_cleanup PASSED [ 92%]
test_matrix_multiplication.py::test_config_loading PASSED [100%]

===== 14 passed in 483.33s (0:08:03) =====
```

Output from executing the pytest test suite.

- **What it Shows:** The screenshot displays the initiation of the test session, identifying the Python environment and installed plugins. It then lists each test collected from `test_matrix_multiplication.py` and reports its status (PASSED). Finally, it summarizes the results, confirming that all 14 tests passed and indicating the total time taken for the test suite execution (approximately 8 minutes in this case).
- **Benefit:** This output provides critical confirmation that the core logic of all implemented matrix multiplication methods, including the complex parallel and shared memory versions, behaves correctly according to the defined test cases (as detailed in Section 7). It verifies error handling for invalid inputs and confirms the functionality of helper components like configuration loading and resource cleanup. Passing tests build confidence in the reliability of the code and the validity of the performance results obtained during benchmarking.

8.2 Console Output (Benchmark Run & Generated Files)

Executing the main benchmark script (matrix_multiplication.py or the GUI's "Run Benchmark" action) produces informational output directly to the console.

```
Starting Matrix Multiplication Benchmarks...
=====

Loading configuration...
Configuration loaded: {'test_sizes': [100, 250, 500, 1000], 'block_size': 32, 'verify_size': 64, 'worker_
counts': [1, 2, 4], 'log_level': 'INFO', 'output_dir': 'output'}

Running performance evaluation...

Performance evaluation completed successfully.
Generating performance plots...
C:\Users\yoga\OneDrive\Desktop\Coding\Parallel\matrix_multiplication.py:965: MatplotlibDeprecationWarning:
The seaborn styles shipped by Matplotlib are deprecated since 3.6, as they no longer correspond to the styl
es shipped by seaborn. However, they will remain available as 'seaborn-v0_8-<style>'. Alternatively, directl
y use the seaborn API instead.
  plt.style.use("seaborn")
Performance plots saved to: output\performance_comparison_20250418_183132.png
Performance results saved to: output\performance_results_20250418_183132.json

Saving results...
Results saved to output\results.json

Benchmark complete!
=====
```

Console output during a typical benchmark run.

- **What it Shows:**

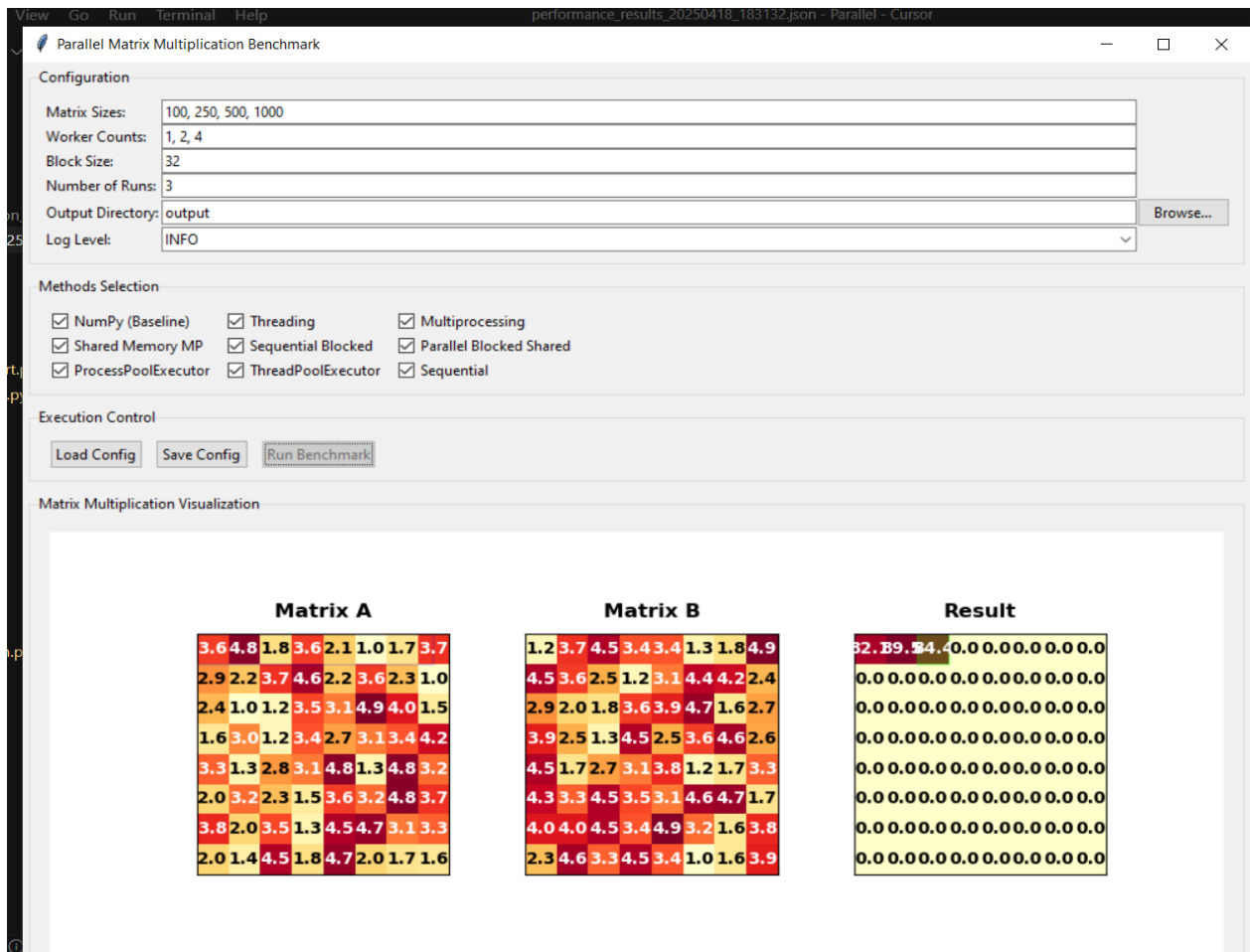
- 1- Initiation messages ("Starting Matrix Multiplication Benchmarks...").
- 2- Confirmation of configuration loading, often displaying the loaded parameters.
- 3- Status updates ("Running performance evaluation...", "Generating performance plots...", "Saving results...").
- 4- File paths where the generated plots (performance_comparison_*.png) and results (performance_results_*.json, results.json) are saved.
- 5- A final confirmation message ("Benchmark complete!").
- 6- (Informational: In this specific run, a MatplotlibDeprecationWarning regarding seaborn styles is visible, which doesn't affect functionality but suggests updating the style usage in the plotting code for future compatibility).

- **Benefit:** This console log provides real-time feedback on the benchmarking process. It allows the user to monitor the different stages, verify the configuration being used, and easily locate the output files containing the detailed results and visualizations. It's essential for transparency and debugging during execution.
- **Generated Files Analysis:** The console output confirms the generation of the following crucial files, which directly fulfill project requirements:
 - 1- **performance_comparison_[timestamp].png:** This PNG file contains the visual plots (Execution Time vs. Matrix Size and Speedup vs. Matrix Size) as shown and discussed in Section 5.2. This is a key deliverable for visualizing performance trends.
 - 2- **performance_results [timestamp].json / results.json:** These JSON files store the detailed, raw performance data gathered during the benchmark. Analyzing the provided performance_results_*.json file confirms it contains:
 - sizes: List of matrix sizes tested.
 - workers: List of worker counts used (including 'single').
 - methods: A nested dictionary structure containing the average execution time for each combination of method, matrix size, and worker count.
 - cpu_usage: A dictionary containing the average CPU utilization percentage recorded for each specific test run combination, keyed descriptively (e.g., "Multiprocessing_1000_4").
 - 3- **Benefit:** The PNG file provides the required visual analysis. The JSON files provide a persistent, machine-readable record of the quantitative results, essential for detailed analysis, reproducibility, and potentially sharing or processing data further. The structure matches the requirements for analyzing execution time, speedup (calculable from the times), CPU utilization, and scalability (by comparing across sizes and workers). All necessary data points for the performance analysis section appear to be present in this JSON output.

8.3 GUI Output

The Graphical User Interface provides interactive output and visualization.

The main Graphical User Interface (GUI) showing configuration, method selection, and the matrix visualization panel.



- **What it Shows:**
 - 1- **Interactive Configuration:** Users see and modify parameters like Matrix Sizes, Worker Counts, etc., in clearly labeled entry fields.
 - 2- **Method Selection:** Checkboxes provide visual confirmation of which multiplication methods are currently selected for the benchmark.
 - 3- **Execution Control:** Buttons (Load/Save Config, Run Benchmark) provide clear action points.
 - 4- **Matrix Visualization Panel:** (As shown in the screenshot) This panel displays matrices A, B, and the Result. It uses color-coding (heatmap) to represent values and directly annotates the numerical values onto the cells for smaller matrices. This provides an intuitive visual representation of the data involved. (During animation, this panel would also show progress highlighting and status updates, as described in Section 8).
 - 5- **Logging Panel:** (Described in Section 8, visible in a complete GUI view) Displays real-time log messages, mirroring the console output but integrated within the GUI.
- **Benefit:** The GUI output makes the system significantly more user-friendly and accessible compared to a purely command-line tool. The primary benefit lies in its ease of use for configuration and execution. The matrix visualization panel, in particular, offers a unique educational benefit by allowing users to visually step through and understand the underlying process of matrix multiplication, which can be abstract when only looking at code or performance numbers.

9. Conclusion

This project successfully implemented and evaluated a diverse set of parallel programming techniques for optimizing matrix multiplication in Python. We compared sequential, NumPy-based, multithreaded, and various multiprocessing approaches (including shared memory and process pools), alongside cache-aware blocked algorithms. A comprehensive benchmarking framework and an interactive GUI were developed to facilitate analysis and visualization.

Our key finding was that while the implemented parallel techniques (multiprocessing, shared memory, parallel blocked) demonstrated parallel execution and scalability *relative to themselves*, they did not outperform the highly optimized NumPy baseline for matrix sizes up to 1000x1000, primarily due to Python's inherent parallel overheads (IPC, serialization, GIL) compared to NumPy's reliance on efficient, low-level C/Fortran libraries. Shared memory offered an improvement over standard multiprocessing but wasn't sufficient to bridge the gap.

Despite not beating the NumPy baseline in speed within the tested range, the project provided valuable insights into:

- The practical challenges and overheads associated with different parallel programming paradigms in Python.
- The critical importance of optimized libraries like NumPy for numerical tasks.
- Techniques for managing resources like shared memory.
- Strategies for task decomposition (chunking, blocking).
- The complexities of integrating backend computations with a responsive GUI.

The project reinforced best practices in modular software design, configuration management, logging, testing, and performance visualization. Future work could explore much larger matrix sizes, investigate C extensions (Cython) or Numba for potentially lower overhead parallelism, or delve deeper into GPU acceleration using libraries like CuPy.